1. Introduction

2. Overview of the Script

3. Technology Stack

4. Execution Flow

5. Grade Calculation

6. Sample Outputs

7. Future Enhancements

8. Conclusion

Introduction

This document presents a comprehensive overview and explanation of a Python script developed to create professional PDF reports using SonarQube metrics. The script streamlines data collection, visualization, and polished reporting to support continuous code quality assessment. It is valuable for both internal teams and external stakeholders, providing a clear and accessible snapshot of code health.

Overview of the Script

The SonarQube Automated Report Generator is a Python-based tool designed to seamlessly connect to a SonarQube server, extract essential project metrics, visualize the data using intuitive charts, and compile all insights into a well-structured, professionally designed PDF report.

Purpose

The main goal of the script is to:

- Automatically extract key software quality metrics from SonarQube
- Enhance data clarity through easy-to-understand visualizations
- Generate a comprehensive PDF report that includes:
    - Code smells, bugs, and vulnerabilities
    - Code coverage and duplication insights
    - Quality gate status and rating grades
    - A categorized list of current issues

High-Level Flow

- **Input Parsing** – Accepts host URL, project key, and authentication token via command-line input
- **Data Collection** – Retrieves metrics, issues, project information, and quality gate status using SonarQube APIs
- **Data Processing** – Transforms raw JSON responses into structured tables and insightful charts
- **Report Generation** – Compiles the processed data into a multi-section PDF including summaries, metrics, visualizations, and a detailed issue breakdown
- **Output** – Produces a polished, self-contained PDF report that can be easily shared with stakeholders

Why Use This Script

- Saves significant time compared to manual dashboard analysis
- Delivers an offline, portable format of critical quality metrics
- Makes it easier for non-technical stakeholders to grasp project health
- Simple to integrate into CI/CD pipelines or schedule for automated reporting

Technology Stack

- **Library**

requests	Makes HTTP calls to SonarQube REST API to fetch metrics and issue data
pandas	Aggregates and analyzes issue types; used for pie chart generation
matplotlib	Creates bar and pie charts to visualize code quality metrics
reportlab	Builds the PDF report with tables, styles, and images
argparse	Parses command-line arguments for user inputs like project key, token
os	Handles directory creation for storing chart images
datetime	Adds timestamp to report
concurrent.features	Enables concurrent data fetching using threads for performance

- **Modules / Functions**

fetch_metrics	Retrieves quality metrics such as bugs, vulnerabilities, coverage, etc.
fetch_project_info()	Gets project metadata like project key, name, etc.
fetch_quality_gate_status()	Checks quality gate pass/fail status
fetch_quality_gate_conditions()	Fetches quality gate conditions or thresholds
fetch_issues()	Retrieves all current issues (OPEN, REOPENED, CONFIRMED)
generate_charts()	Generates bar and pie charts from metrics and issues
generate_pdf()	Creates the final PDF report with all data and charts
add_page_number()	Adds page numbers to each page of the report

- **Input (Data Flow)**

--host	URL of the SonarQube server
--project	Project key to identify the target project in SonarQube
--token	Authentication token for API access

Authenctication (API Access)

Basic Authentication :-

auth = (token, “ ”)

Custom headers for API Calls:-

headers = {“Accept”: “application/json”}

Execution Flow

1. The script accepts inputs for SonarQube server URL, project key, authentication token, etc and navigates to SonarQube Dashboard.

2. It then calculates the metrics and data using the ThreadPoolExecutor.

3. Using the libraries and functions it generates other metadata and charts.

4. After all those processing, the ReportLab of Python generates a PDF Report .

Grade Calculation

The script assesses the project's quality based on three key SonarQube metrics:

- **Bugs** correspond to **Reliability**, measured by the **reliability_rating** metric.
- **Vulnerabilities** correspond to **Security**, measured by the **security_rating** metric.
- **Code Smells** correspond to **Maintainability**, measured by the **sqale_rating** metric.

Each of these metrics returns a numeric rating from 1 to 5, where:

- 1 represents the highest quality level (best).
- 5 represents the lowest quality level (worst).

The script converts these numeric ratings into letter grades as follows:

- 1 → A
- 2 → B
- 3 → C
- 4 → D
- 5 → E

Sample Outputs

SonarQube Report - [redacted]

Generated on: 2025-06-06 08:08:47

Quality Gate Status: PASS

Metric	Value
Bugs	0
Vulnerabilities	5
Code Smells	6
Coverage	-
Duplication	0.0%

Overall Grade:

Metric	Value
Bugs	0
Code Smells	6
Duplication	0.0%
Vulnerabilities	5
Reliability	A
Security	E
Maintainability	A

Charts:

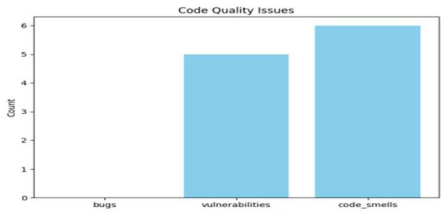

Figure 1: Code Quality Issues Overview

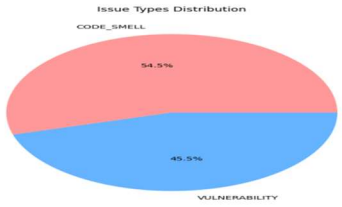

Figure 2: Issue Types Distribution

Issues (11):

Key	Severity	Type	Message	Component	Line	Status
33cff819-1055-4803-b03f-1e2c68bfacf6	MAJOR	VULNERABILITY	Bind this resource's automounted service account to RBAC or disable automounting.	[redacted]	37	OPEN
9fc8be0c-e086-4024-9afa-ca6fce196562	MAJOR	CODE_SMELL	Specify a memory request for this container.		38	OPEN
b7e64943-1620-4686-a29a-ec1d33f3c703	MAJOR	CODE_SMELL	Specify a CPU request for this container.		38	OPEN
daca3916-6f61-405d-93f2-b000d5735e45	MAJOR	VULNERABILITY	Specify a memory limit for this container.		38	OPEN
e306603d-615e-463a-91b8-ec3dc30c2859	MAJOR	CODE_SMELL	Specify a storage request for this container.		38	OPEN
bf4081df-2828-471c-be54-771448258643	MAJOR	VULNERABILITY	Bind this resource's automounted service account to RBAC or disable automounting.		37	OPEN
9303426f-4046-40c4-a24a-b59a6f2e3fe9	MAJOR	CODE_SMELL	Specify a CPU request for this container.		38	OPEN
ec6d9d64-b32b-4d42-a538-0f7c3a97918e	MAJOR	CODE_SMELL	Specify a storage request for this container.		38	OPEN
f171aaeb-8808-4bc1-8df4-8bb02155777e	MAJOR	CODE_SMELL	Specify a memory request for this container.		38	OPEN
fd036c66-7e25-44b3-9eb7-aacb96e6baa8	MAJOR	VULNERABILITY	Specify a memory limit for this container.		38	OPEN
a809d795-f8c6-42de-a8bf-8b2ad55c6890	BLOCKER	VULNERABILITY	Make sure this AWS Secret Access Key gets revoked, changed, and removed from the code.		11	OPEN

Future Enhancements

Email reporting
Historical Trend Charts

Conclusion

This script provides an efficient and automated way to extract comprehensive code quality data from SonarQube and generate a detailed, visually rich PDF report. By consolidating key metrics such as bugs, vulnerabilities, code smells, and quality gate statuses, along with issue details and graphical representations, it offers valuable insights into the health and maintainability of software projects. The use of concurrent requests improves data fetching performance, and the structured report format facilitates easy analysis and decision-making. With potential enhancements like multi-project support, trend analysis, and integration with CI/CD pipelines, this script can be further expanded to support advanced quality management workflows, helping teams maintain high standards of code quality and security.